

IGCSE to A-Level Bridge Document — VPA Computer Science

IGCSE to A-Level Bridge Document

IGCSE计算机科学 (0478) 到A-Level计算机科学 (9618) 衔接指南 / Victoria Park Academy

1. Overview / 概述

This bridge document is designed for students transitioning from Cambridge IGCSE Computer Science (0478) to Cambridge International A-Level Computer Science (9618). It maps prior knowledge to new requirements, highlights skills progression, and provides a structured transition plan for the summer before Year 12.

本衔接文件旨在帮助学生从剑桥IGCSE计算机科学(0478)过渡到剑桥国际A-Level计算机科学(9618)。它将先前学习的知识映射到新的课程要求，突出技能进阶，并为12年级前的暑假提供结构化的过渡计划。

Key Message / 关键信息: A-Level Computer Science is not simply "more of the same." It demands deeper theoretical understanding, more sophisticated programming, and the ability to apply knowledge to unfamiliar contexts. Students who consolidate IGCSE fundamentals over the summer start Year 12 with confidence.

A-Level计算机科学不仅仅是"更多相同的内容"。它要求更深入的理论理解、更复杂的编程能力，以及将知识应用于陌生情境的能力。那些在暑假巩固IGCSE基础知识的学生将以自信的状态开始12年级的学习。

2. Topic-by-Topic Mapping / 主题对照表

The table below shows how each IGCSE topic connects to A-Level content. 下表展示了每个IGCSE主题如何与A-Level内容相连接。

2.1 Theory Topics / 理论主题

IGCSE Topic (0478)	A-Level Topic (9618)	Progression / 进阶	Priority / 优先级
1.1 Binary systems & hex (number bases, conversions, uses)	1.1 Information representation (binary, BCD, ASCII, Unicode, bitmaps, sound, video compression)	From simple conversion to understanding why different representations exist and how compression algorithms work.	High
1.2 Data transmission (types, methods, error detection: parity, checksum, ARQ)	1.3 Communication & Internet technologies (packet switching, protocols, TCP/IP stack, encryption)	From basic error detection to full protocol stacks, encryption, and network security principles.	High
1.3 Hardware (CPU components, von Neumann, fetch-decode-execute, I/O, storage)	1.2 Hardware (processor architecture, registers, assembly language, Boolean algebra, logic gates, flip-flops, embedded systems)	From conceptual CPU to register-level detail, assembly programming, and sequential logic circuits.	Critical

IGCSE Topic (0478)	A-Level Topic (9618)	Progression / 进阶	Priority / 优先级
1.4 Software (system vs application, OS functions)	1.4 System software (compilers, interpreters, assemblers, linkers, loaders, OS memory management)	From OS functions to understanding how high-level code becomes machine code; memory management algorithms.	High
1.5 Data representation (binary, hex, ASCII, file sizes)	1.1 (extended) + 3.1 Data representation in depth (floating-point, normalisation, two's complement arithmetic)	From integer representation to floating-point, normalisation, and binary arithmetic operations.	Critical
2.1 Algorithm design (pseudocode, flowcharts, trace tables)	2.1 Algorithm design (stepwise refinement, modular design, recursion, Big-O complexity)	From simple algorithms to recursive solutions, complexity analysis, and structured decomposition.	Critical
2.2 Programming (sequence, selection, iteration, arrays, file handling)	2.2 Programming (OOP, classes, inheritance, polymorphism, file handling, exception handling)	From procedural to object-oriented programming; exception handling and more complex data structures.	Critical
2.3 Databases (flat file, relational, SQL, normalisation basics)	2.3 Databases (3NF, BCNF, referential integrity, complex queries, E-R diagrams)	From basic SQL to full normalisation theory, E-R modelling, and database design principles.	Medium
2.4 Boolean logic (truth tables, logic gates, simplification)	1.2 Boolean algebra (Karnaugh maps, De Morgan's laws, simplification, circuit design)	From truth tables to algebraic simplification, Karnaugh maps, and sequential circuit design.	High
3.1 Ethics, security, AI (basic concepts)	4.1 Security, privacy, ethics (encryption standards, legislation, AI ethics, professional responsibility)	From awareness to deep analysis of legislation, encryption standards, and professional codes of conduct.	Medium

2.2 Practical Skills / 实践技能

IGCSE Skill	A-Level Skill	Progression / 进阶
Programming in Python (procedural)	Programming in Python (OOP, file I/O, exception handling)	Must master classes, objects, inheritance, and exception handling before Year 12.
Simple trace tables	Complex trace tables with recursion, nested loops, and OOP	Practice tracing recursive algorithms and object method calls.
Basic SQL (SELECT, INSERT, UPDATE, DELETE)	Advanced SQL (JOINS, nested queries, aggregate functions, DDL)	Revise all SQL basics; learn JOINS and aggregate functions over summer.
Writing pseudocode for simple problems	Writing pseudocode for complex, modular systems	Practice decomposing problems into modules with clear interfaces.
Understanding test data (normal, boundary, erroneous)	Designing comprehensive test plans with equivalence partitioning	Learn equivalence partitioning and how to justify test case selection.

3. Skills Progression: What's New vs. What's Deepened / 技能进阶：新增与深化

3.1 What is New at A-Level? / A-Level新增内容

Object-Oriented Programming (OOP) — classes, objects, inheritance, polymorphism, encapsulation.
面向对象编程——类、对象、继承、多态、封装。

Recursion — recursive algorithms, base cases, recursive case design, stack frames.
递归——递归算法、基准情形、递归情形设计、栈帧。

Big-O Complexity — time and space complexity, comparing algorithm efficiency.
大O复杂度——时间和空间复杂度，比较算法效率。

Assembly Language — low-level programming, register operations, addressing modes.
汇编语言——低级编程、寄存器操作、寻址模式。

Floating-Point Representation — IEEE 754, mantissa and exponent, normalisation. 浮点数表示——IEEE 754标准、尾数和指数、规范化。

Network Protocols in Depth — TCP/IP stack, packet switching, encryption protocols.
网络协议深入——TCP/IP协议栈、分组交换、加密协议。

Database Normalisation — 1NF, 2NF, 3NF, BCNF with formal proofs.
数据库规范化——第一至第三范式、BC范式及形式化证明。

Computer Security & Encryption — symmetric/asymmetric encryption, digital signatures, certificates, hashing.
计算机安全与加密——对称/非对称加密、数字签名、证书、哈希。

Artificial Intelligence — machine learning basics, neural networks, AI ethics.
人工智能——机器学习基础、神经网络、AI伦理。

System Software — compilers, interpreters, assemblers, linkers, loaders, memory management.
系统软件——编译器、解释器、汇编器、链接器、加载器、内存管理。

3.2 What is Deepened? / 深化内容

Binary Arithmetic — from simple conversion to addition, subtraction, two's complement overflow, shift operations.
二进制运算——从简单转换到加减法、补码溢出、移位操作。

Logic Design — from truth tables to Karnaugh maps, De Morgan's laws, and sequential circuits (flip-flops).
逻辑设计——从真值表到卡诺图、德摩根定律和时序电路（触发器）。

Algorithm Design — from simple pseudocode to stepwise refinement, modular design, and complexity analysis.
算法设计——从简单伪代码到逐步求精、模块化设计和复杂度分析。

Data Structures — from 1D arrays to 2D arrays, stacks, queues, linked lists, trees, graphs.
数据结构——从一维数组到二维数组、栈、队列、链表、树、图。

Programming — from procedural scripts to structured OOP programs with file handling and exception handling.
编程——从过程式脚本到具有文件处理和异常处理的结构化OOP程序。

Systems Architecture — from conceptual CPU to register-level detail, instruction sets, and assembly language.
系统架构——从概念性CPU到寄存器级细节、指令集和汇编语言。

Warning / 警告: Students who achieved a high grade at IGCSE through memorisation alone often struggle at A-Level. A-Level requires application of knowledge to novel problems. The summer transition work (Section 5) is designed to develop this skill.

仅通过死记硬背在IGCSE取得高分的学生往往在A-Level遇到困难。A-Level要求将知识应用于新问题。暑假过渡作业（第5节）旨在培养这一能力。

4. Prerequisite Knowledge Checklist / 先修知识检查清单

Before starting A-Level Computer Science, students should be confident in ALL of the following.
在开始A-Level计算机科学之前，学生应对以下内容全部掌握。

4.1 Number Systems & Data Representation / 数制与数据表示

Convert between binary, denary, and hexadecimal confidently. 熟练进行二进制、十进制和十六进制之间的转换。

Understand binary addition and the concept of overflow. 理解二进制加法和溢出概念。

Know how ASCII represents characters and why Unicode is needed. 了解ASCII如何表示字符以及为什么需要Unicode。

Calculate file sizes for bitmap images (resolution $\times$ colour depth). 计算位图图像的文件大小(分辨率 $\times$ 颜色深度)。

Understand the difference between analogue and digital data. 理解模拟数据与数字数据的区别。

4.2 Computer Architecture / 计算机体系结构

Name and describe the function of the ALU, CU, MAR, MDR, PC, and accumulator. 说出并描述ALU、CU、MAR、MDR、PC和累加器的功能。

Explain the fetch-decode-execute cycle with a diagram. 用图示解释取指-译码-执行周期。

Understand the von Neumann architecture and stored program concept. 理解冯·诺依曼架构和存储程序概念。

Describe the purpose of cache, RAM, and ROM. 描述缓存、RAM和ROM的用途。

4.3 Programming Fundamentals / 编程基础

Write programs using sequence, selection (IF...ELSE), and iteration (FOR, WHILE, REPEAT). 使用顺序、选择(IF...ELSE)和迭代(FOR、WHILE、REPEAT)编写程序。

Use 1D arrays effectively (declaration, assignment, traversal). 有效使用一维数组(声明、赋值、遍历)。

Write and call functions/procedures with parameters and return values. 编写和调用带参数和返回值的函数/过程。

Perform file operations: open, read, write, close text files. 执行文件操作: 打开、读取、写入、关闭文本文件。

Validate user input and handle invalid data gracefully. 验证用户输入并优雅地处理无效数据。

4.4 Algorithms & Problem-Solving / 算法与问题解决

Trace simple algorithms using a trace table. 使用跟踪表跟踪简单算法。

Identify and correct logic errors in given code. 识别并修正给定代码中的逻辑错误。

Explain the difference between linear search and binary search. 解释线性搜索和二分搜索的区别。

Describe bubble sort and insertion sort with examples. 用示例描述冒泡排序和插入排序。

Understand the concept of abstraction and decomposition. 理解抽象和分解的概念。

4.5 Databases / 数据库

Write SQL SELECT statements with WHERE clauses. 编写带WHERE子句的SQL SELECT语句。

Understand the difference between flat-file and relational databases. 理解平面文件数据库和关系数据库的区别。

Explain primary keys and foreign keys. 解释主键和外键。

Know the purpose of normalisation (removing redundancy). 了解规范化的目的(消除冗余)。

4.6 Networks & Security / 网络与安全

Describe the client-server model. 描述客户端-服务器模型。

Explain how parity and checksum detect errors. 解释奇偶校验和校验和如何检测错误。

List common threats (malware, phishing, DDoS) and basic protections. 列出常见威胁(恶意软件、钓鱼、DDoS)和基本防护措施。

Understand the difference between the Internet and the World Wide Web. 理解互联网和万维网的区别。

Action Required / 需要行动: If you cannot confidently tick ALL items in this checklist, you MUST complete the summer transition work (Section 5) before starting Year 12. Speak to your teacher if you need support.

如果您不能自信地勾选此清单中的所有项目，您必须在开始12年级之前完成暑假过渡作业（第5节）。如需支持，请与您的老师联系。

5. Summer Transition Work / 暑假过渡作业

The following structured programme should be completed over 6 weeks (approximately 4-5 hours per week). 以下结构化计划应在6周内完成（每周约4-5小时）。

Week 1: Consolidate IGCSE Fundamentals / 巩固IGCSE基础

Revise all number system conversions (binary ☐ denary ☐ hex) — 50 practice questions. 复习所有数制转换（二进制 ☐ 十进制 ☐ 十六进制）——50道练习题。

Revise binary addition and two's complement representation. 复习二进制加法和补码表示。

Review the fetch-decode-execute cycle and draw a detailed diagram. 复习取指-译码-执行周期并绘制详细图示。

Week 2: Programming Refresher / 编程复习

Write a Python program that reads a text file, processes the data, and writes results to a new file. 编写一个Python程序，读取文本文件，处理数据，并将结果写入新文件。

Implement a simple menu system using functions. 使用函数实现一个简单的菜单系统。

Practice writing pseudocode for 5 different problems (search, sort, validation). 为5个不同的问题（搜索、排序、验证）练习编写伪代码。

Week 3: Introduction to OOP / 面向对象编程入门

Complete the free SoloLearn Python OOP course (or equivalent). 完成免费的SoloLearn Python OOP课程（或同等课程）。

Design a Student class with attributes (name, age, grades) and methods (add_grade, get_average). 设计一个Student类，包含属性（姓名、年龄、成绩）和方法（添加成绩、获取平均分）。

Create a BankAccount class with deposit, withdraw, and display_balance methods. 创建一个BankAccount类，包含存款、取款和显示余额方法。

Week 4: Data Structures & Algorithms / 数据结构与算法

Implement bubble sort and insertion sort in Python; compare their performance on large datasets. 用Python实现冒泡排序和插入排序；比较它们在大型数据集上的性能。

Implement linear search and binary search; understand when each is appropriate. 实现线性搜索和二分搜索；理解何时使用每种方法。

Learn about stacks and queues; implement a stack using a Python list. 学习栈和队列；使用Python列表实现栈。

Week 5: Databases & SQL / 数据库与SQL

Install SQLite and practice writing SELECT, INSERT, UPDATE, DELETE statements. 安装SQLite并练习编写SELECT、INSERT、UPDATE、DELETE语句。

Learn JOINS (INNER JOIN, LEFT JOIN) with at least 10 practice queries. 学习JOIN (INNER JOIN, LEFT JOIN)，至少练习10个查询。

Design a simple database schema for a school library system (3+ tables). 为学校图书馆系统设计一个简单的数据库模式（3个以上表）。

Week 6: Preview A-Level Content / 预习A-Level内容

Read Chapter 1 of the A-Level textbook (Information Representation). 阅读A-Level教科书第1章（信息表示）。

Watch Cambridge-recommended videos on floating-point representation. 观看剑桥推荐的关于浮点数表示的视频。

Attempt 3 past A-Level Paper 1 questions (do not worry about getting them right — focus on understanding the style). 尝试3道A-Level试卷1的历年试题（不必担心答对——重点是理解题型风格）。

Submission / 提交: Bring evidence of completed summer work (code files, screenshots, notes) to your first A-Level lesson. Your teacher will review it and provide feedback.

将完成的暑假作业证据（代码文件、截图、笔记）带到您的第一节A-Level课程。您的老师将审阅并提供反馈。

6. Recommended Reading / 推荐阅读

6.1 Textbooks / 教科书

Cambridge International AS & A Level Computer Science Coursebook by Sylvia Langfield and Dave Duddell (Hodder Education) — Essential / 必备

Cambridge International AS & A Level Computer Science Revision Guide by Helen Williams (Hodder Education) — Highly recommended / 强烈推荐

Computer Science: An Overview by J. Glenn Brookshear and Dennis Brylow (Pearson) — Excellent for breadth / 拓展阅读 excellent

6.2 Supplementary Books / 补充读物

Code: The Hidden Language of Computer Hardware and Software by Charles Petzold — Brilliant explanation of how computers work from first principles / 从第一性原理解释计算机工作原理的杰作

Grokking Algorithms by Aditya Bhargava — Visual, intuitive introduction to algorithms / 直观易懂的算法入门

Clean Code by Robert C. Martin — For students who want to write professional-quality code / 希望编写专业级代码的学生

The Pragmatic Programmer by Andrew Hunt and David Thomas — Broader software engineering perspective / 更广泛的软件工程视角

Algorithms to Live By by Brian Christian and Tom Griffiths — Accessible, engaging link between CS and everyday life / 计算机科学与日常生活的有趣联系

6.3 Chinese-Language Resources / 中文资源

《计算机科学导论》（原书第4版）—— 布鲁克希尔著，全面覆盖计算机科学基础

《算法图解》—— 算法入门经典，配有丰富图示

《编码：隐匿在计算机软硬件背后的语言》—— Charles Petzold 著，中文版可读性极佳

《Python编程：从入门到实践》—— Eric Matthes 著，适合巩固Python基础

7. Online Resources / 在线资源

7.1 Free Learning Platforms / 免费学习平台

Resource / 资源	Content / 内容	Best For / 最适合
Cambridge International	Syllabus, specimen papers, examiner reports	Official syllabus understanding / 理解官方大纲
Zakonweb	A-Level CS theory notes, quizzes, past paper solutions	Theory revision / 理论复习
SoloLearn	Interactive Python, C++, Java courses	Programming practice / 编程练习
Codecademy	Structured programming courses (free tier available)	OOP and Python deep dive / OOP和Python深入学习
Khan Academy	Algorithms, cryptography, Internet protocols	Visual explanations / 可视化解释
W3Schools	SQL tutorials, Python reference	Quick syntax lookup / 快速语法查询
CS Unplugged	Activities teaching CS concepts without computers	Understanding fundamentals / 理解基础概念
VisuAlgo	Animated visualisations of data structures and algorithms	Algorithm visualisation / 算法可视化
Computerphile (YouTube)	Expert explanations of CS topics by university academics	Deep understanding / 深入理解
CrashCourse Computer Science	40-episode comprehensive CS overview	Broad overview / 广泛概览

7.2 Programming Practice / 编程练习

- HackerRank — Structured challenges in Python, algorithms, and data structures. Python、算法和数据结构的结构化挑战。
- LeetCode — Interview-style problems excellent for developing algorithmic thinking. 面试风格的问题，非常适合培养算法思维。
- Project Euler — Mathematical problems requiring programming solutions. 需要编程解决方案的数学问题。
- Codewars — Gamified coding challenges with community solutions. 带有社区解决方案的游戏化编码挑战。
- Replit — Browser-based IDE; excellent for quick Python experiments. 基于浏览器的IDE；非常适合快速Python实验。

7.3 Chinese-Language Online Resources / 中文在线资源

- 中国大学MOOC（慕课） — 搜索"计算机科学导论"、"Python程序设计"等课程
- B站（哔哩哔哩） — 搜索"CS61A"、"计算机网络"、"数据结构与算法"等优质课程
- 菜鸟教程（runoob.com） — Python、SQL、数据结构等快速参考
- 力扣（LeetCode 中国） — 中文界面的算法练习平台

8. Key Differences: IGCSE vs. A-Level / 关键差异：IGCSE与A-Level

Aspect / 方面	IGCSE (0478)	A-Level (9618)
Exam structure / 考试结构	2 papers (Theory + Practical)	4 papers (AS: 2 papers; A2: 2 papers)
Programming focus / 编程重点	Procedural Python	OOP Python + Pseudocode
Theory depth / 理论深度	Broad, conceptual understanding	Deep, technical detail with application
Mathematical demand / 数学要求	Basic arithmetic and logic	Binary arithmetic, Boolean algebra, complexity analysis

Aspect / 方面	IGCSE (0478)	A-Level (9618)
Problem-solving / 问题解决	Structured, familiar problems	Novel, unfamiliar contexts requiring synthesis
Written communication / 书面表达	Short answers, some explanation	Extended prose, justification, evaluation
Independent study / 独立学习	Moderate	High — students must read beyond the textbook

9. Advice from Former VPA Students / 往届VPA学生建议

"Start OOP in the summer. I didn't, and I spent the first term catching up." — Year 13 student, achieved A*.

"暑假就开始学OOP。我当时没学，结果整个第一学期都在补课。" — 13年级学生，获得A*。

"The jump in theory is bigger than the jump in programming. Don't underestimate the hardware and data representation topics." — Year 13 student, achieved A.

"理论的跳跃比编程的跳跃更大。不要低估硬件和数据表示主题。" — 13年级学生，获得A。

"Do every past paper you can find. The question style is very specific to Cambridge." — Year 13 student, achieved A*.

"尽可能做每一套历年试卷。剑桥的题型非常独特。" — 13年级学生，获得A*。

"Don't just memorise — understand. The exam tests whether you can apply concepts, not recite them." — Year 13 student, achieved A*.

"不要只是记忆——要理解。考试测试的是你是否能应用概念，而不是背诵它们。" — 13年级学生，获得A*。

10. Contact & Support / 联系与支持

If you have questions about the transition to A-Level Computer Science, or if you are concerned about any prerequisite knowledge, please contact:

如果您对过渡到A-Level计算机科学有任何疑问，或者担心任何先修知识，请联系：

Head of Computer Science: cs@victoriaparkacademy.edu

A-Level Coordinator: alevel@victoriaparkacademy.edu

Subject Teacher: Your IGCSE CS teacher will provide handover notes to your A-Level teacher.

Remember / 记住: The bridge from IGCSE to A-Level is challenging but absolutely achievable with preparation. Use this summer wisely, and you will begin Year 12 with confidence and momentum.

从IGCSE到A-Level的过渡充满挑战，但只要做好准备就绝对可以实现。明智地利用这个暑假，您将以自信和动力开始12年级的学习。

☐ Victoria Park Academy | A-Level Computer Science (9618) | IGCSE to A-Level Bridge Document

Document Version 1.0 | Updated: 2025